

Práctica 3 – Realidad aumentada con múltiples Image Targets y movimiento entre marcadores

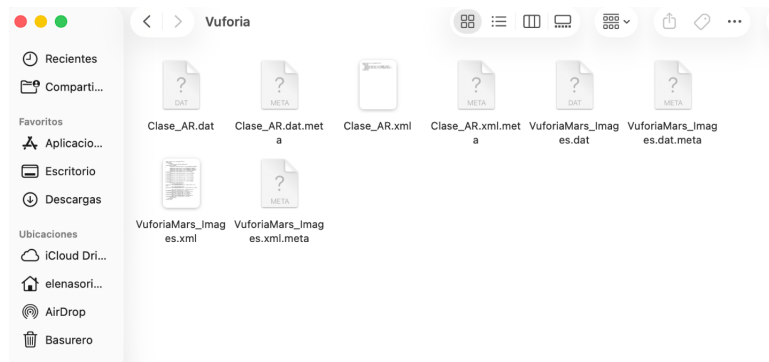
Materiales y herramientas

- MacBook Air con macOS y Unity Hub.
- Unity 2022.3.62f3 LTS.
- Vuforia Engine configurado dentro del proyecto.
- Visual Studio Code para la edición del script C#.
- Xcode para la generación, firma e instalación del proyecto iOS.
- iPhone físico para pruebas de compilación y ejecución.
- Base de datos de Image Targets denominada Clase_AR.
- Cuatro marcadores de imagen y diferentes objetos 3D: modelo de iPhone, cubos y cilindro.
- Materiales de colores creados dentro de Unity para diferenciar visualmente los objetos.

Desarrollo de la práctica

1. Preparación del proyecto y base de Vuforia

Se trabajó sobre el proyecto de la Práctica 3 manteniendo la ARCamera y la configuración de Vuforia. Antes de modificar la escena se verificó que la base de datos de marcadores estuviera disponible en Assets → StreamingAssets → Vuforia y que los archivos correspondientes a Clase_AR se encontraran dentro del proyecto. Esto permitió que los Image Targets pudieran seleccionarse desde el componente Image Target Behaviour.



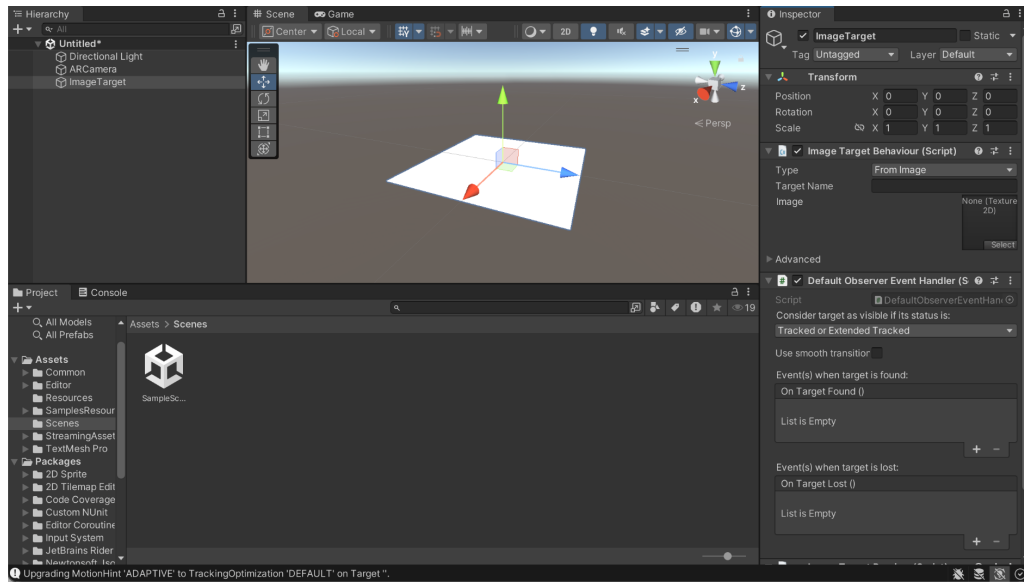
Archivos de la base de datos de Vuforia utilizados en la práctica.

2. Creación y configuración de los Image Targets

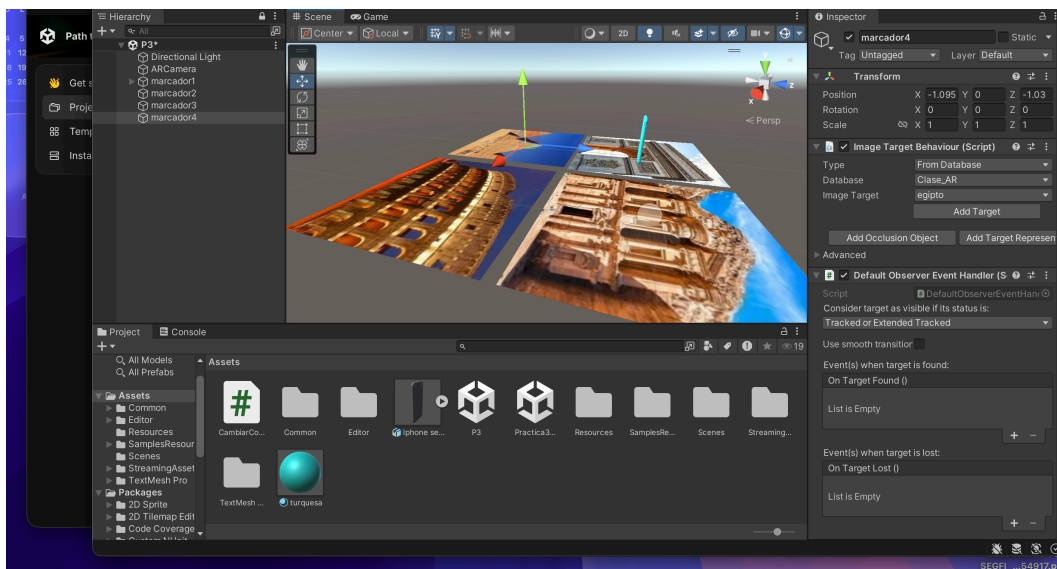
Se agregaron cuatro objetos Image Target en la Hierarchy y se renombraron como marcador1, marcador2, marcador3 y marcador4. Cada marcador se configuró con Type = From Database y Database = Clase_AR. Posteriormente se asignó un Image Target diferente a cada uno para que Vuforia pudiera distinguir las imágenes físicas utilizadas durante la prueba.

1. Crear o duplicar los objetos Image Target necesarios dentro de la escena.

2. Asignar Type = From Database en Image Target Behaviour.
3. Seleccionar la base Clase_AR.
4. Elegir un target diferente para cada marcador.
5. Mantener los objetos 3D como hijos de su marcador correspondiente.



Configuración inicial de un Image Target dentro de Unity.

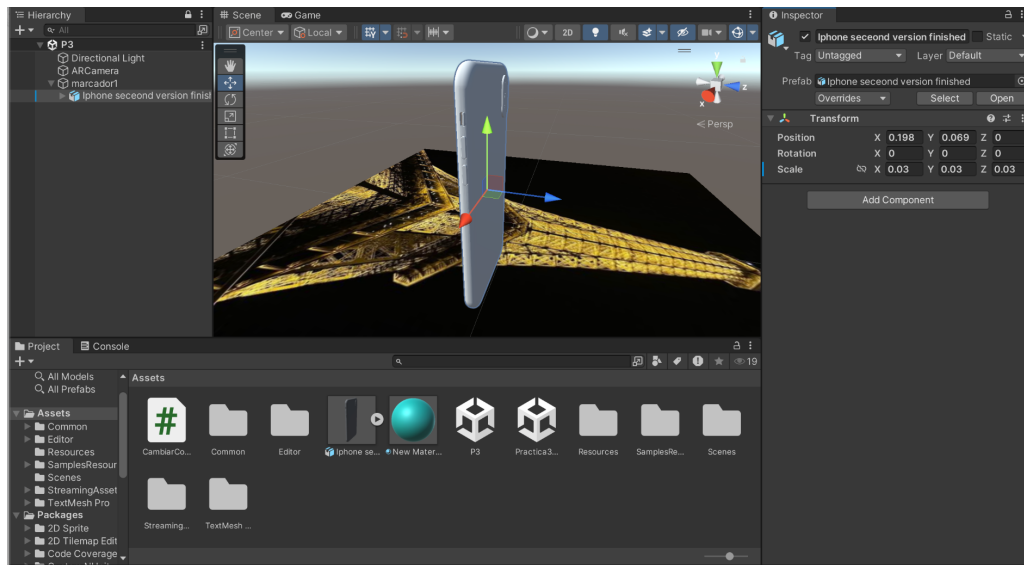


Marcador cargado desde la base Clase_AR.

3. Asociación de objetos 3D a los marcadores

Cada Image Target recibió un objeto 3D como hijo en la Hierarchy. El modelo de iPhone se colocó sobre uno de los marcadores y los demás targets se utilizaron con objetos geométricos de prueba. Para que el contenido apareciera correctamente sobre la imagen, se ajustaron Position, Rotation y Scale de cada objeto respecto a su Image Target, evitando desplazar el marcador completo.

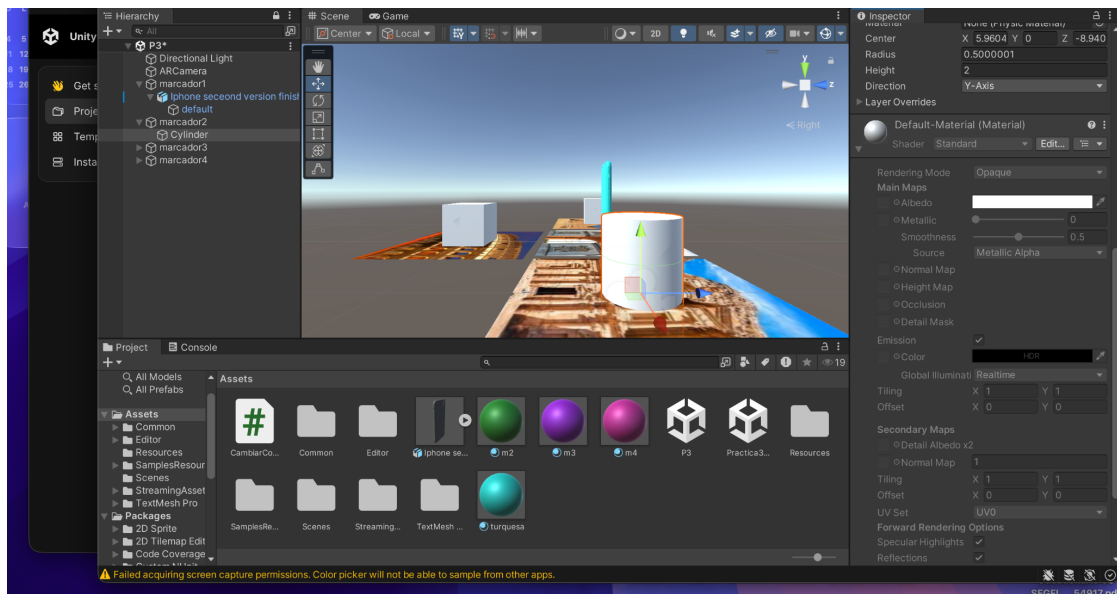
En el caso del modelo de iPhone fue necesario reducir su escala y modificar su posición local hasta colocarlo directamente sobre el plano del marcador. Esta relación padre-hijo garantiza que el objeto conserve la referencia espacial del target cuando Vuforia lo detecta.



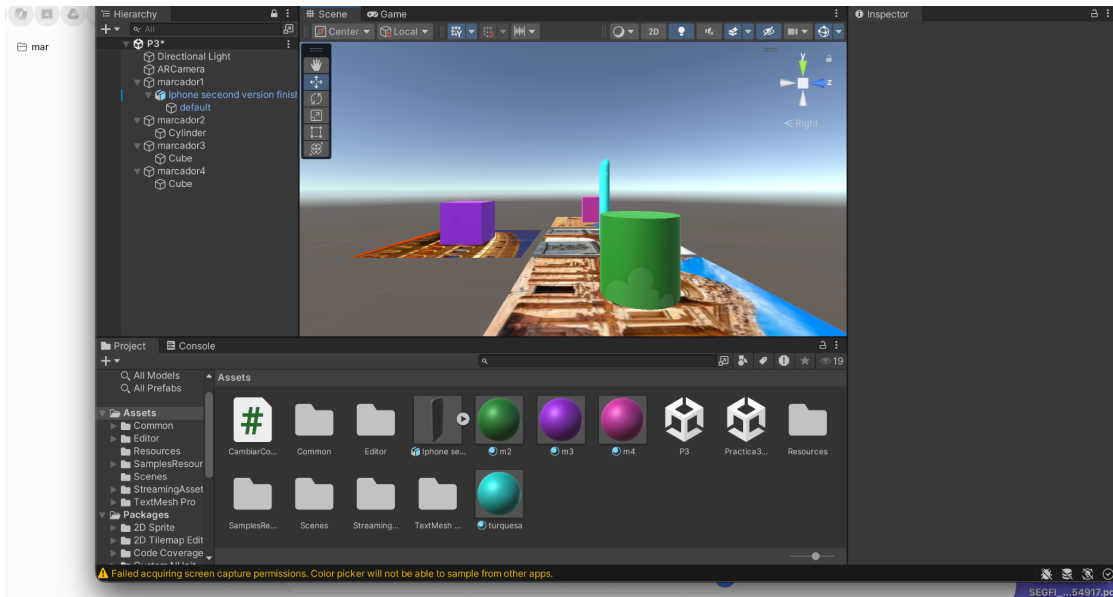
Ajuste del modelo 3D sobre su Image Target.

4. Creación y aplicación de materiales

Para diferenciar visualmente los objetos se crearon materiales independientes desde Assets → Create → Material. A cada material se le asignó un color distinto y posteriormente se arrastró al campo Materials del Mesh Renderer del objeto correspondiente. Se utilizaron, entre otros, tonos turquesa, verde, morado y rosa.



Escena con los distintos materiales preparados para los objetos.

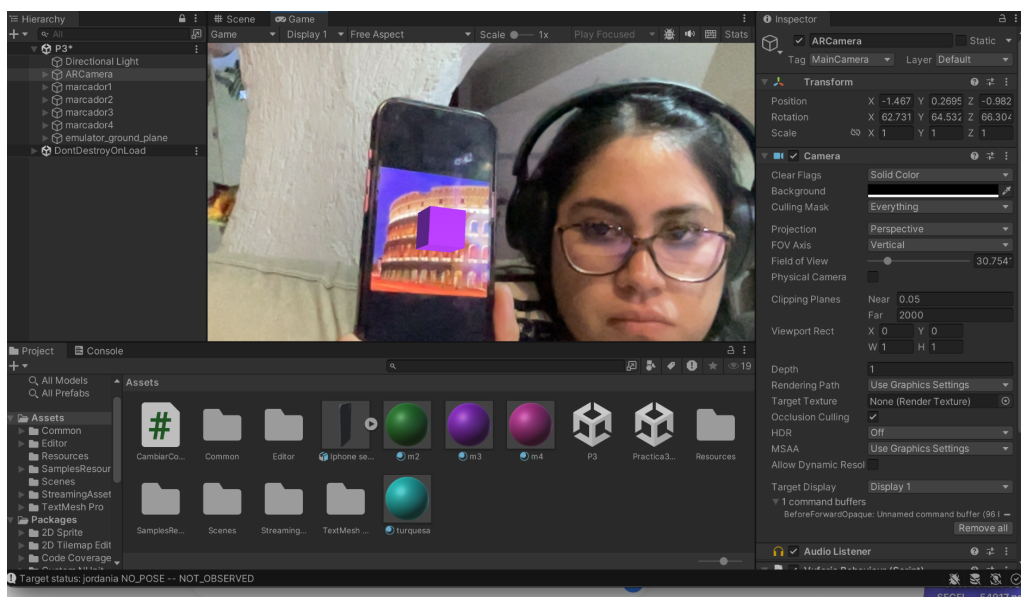


Aplicación de materiales a los objetos asociados a los marcadores.

5. Prueba de reconocimiento de múltiples marcadores

Una vez configurados los cuatro marcadores, se inició Play Mode en Unity. Vuforia utilizó la cámara para reconocer las imágenes y mostrar el objeto 3D asociado. La configuración de Vuforia se ajustó para permitir hasta cuatro Image Targets simultáneamente, con el fin de trabajar con varios marcadores dentro de la misma escena.

Durante la validación en Unity se comprobó el reconocimiento de diferentes targets y la aparición de los objetos configurados, incluidos el cubo y el cilindro. Las pruebas se realizaron mostrando las imágenes frente a la cámara con suficiente iluminación y evitando reflejos.



Prueba de reconocimiento de un marcador con objeto 3D.

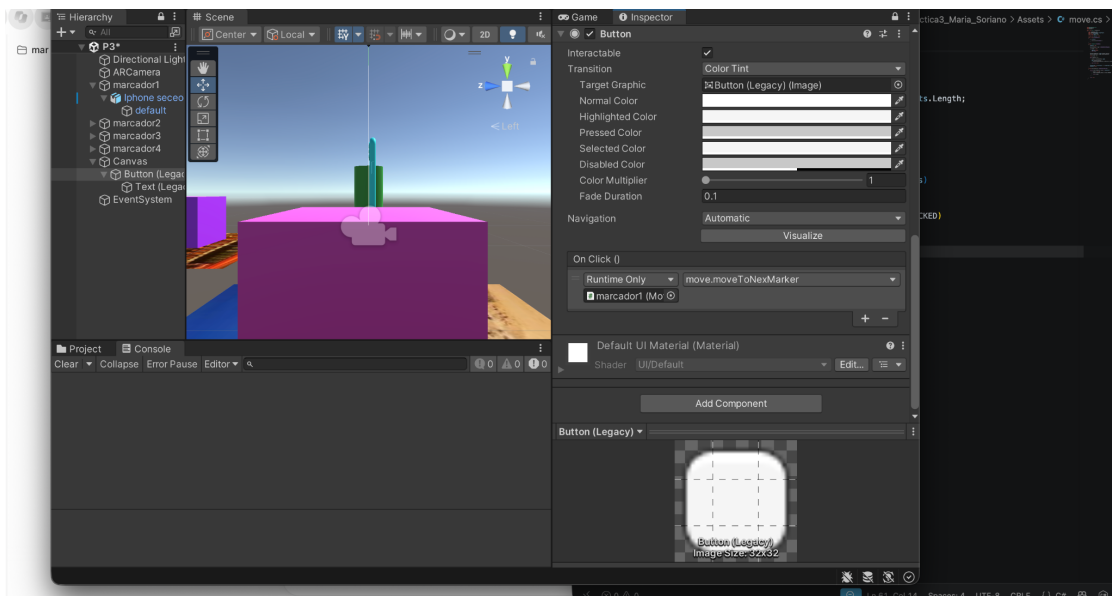


Reconocimiento de otro target durante Play Mode.

6. Creación de la interfaz de usuario

Se agregó un Canvas a la escena y dentro de él un Button (Legacy). El botón se mantuvo en una posición visible de la interfaz y se utilizó como disparador del comportamiento programado. Unity creó también el EventSystem requerido para procesar la interacción del usuario.

El evento On Click() del botón se vinculó con el componente move y con el método público moveToNexMarker(). De esta manera, al presionar el botón se ejecuta la corrutina encargada de mover el modelo hacia un target detectado.



Configuración final del evento On Click() del botón.

7. Script move.cs

Se creó el script move.cs para controlar el desplazamiento del modelo. El script mantiene una referencia al GameObject que debe moverse, un arreglo de ObserverBehaviour para los Image Targets, un índice de control y una velocidad configurable. La función pública moveToNextMarker() inicia una corrutina únicamente cuando no existe otro movimiento en curso.

La corrutina obtiene un target detectado, toma la posición actual del modelo y la posición del marcador destino, y utiliza Vector3.Lerp() junto con Time.deltaTime para realizar una interpolación progresiva. Al terminar, se actualiza el índice y se habilita nuevamente la posibilidad de ejecutar otro desplazamiento.

```
move.cs
Users > elenasorianobarrera > Desktop > 2027-1 > Tems Selectos 3 > Practica3_María_Soriano > Assets > move.cs > ...
1  using System.Collections;
2  using UnityEngine;
3  using Vuforia;
4
5  0 references
6  public class move : MonoBehaviour
7  {
8      2 references
9      public GameObject model;
10     2 references
11     public ObserverBehaviour[] ImageTargets;
12     2 references
13     public int currentTarget;
14     1 reference
15     public float speed = 1.0f;
16
17     4 references
18     private bool isMoving = false;
19
20     0 references
21     public void moveToNexMarker()
22     {
23         if (!isMoving)
24         {
25             StartCoroutine(MoveModel());
26         }
27     }
28
29     1 reference
30     private IEnumerator MoveModel()
31     {
32         isMoving = true;
33
34         ObserverBehaviour target = GetNextDetectedTarget();
35
36         if (target == null)
37         {
38             isMoving = false;
39             yield break;
40         }
41     }
```

```
34 Vector3 startPosition = model.transform.position;
35 Vector3 endPosition = target.transform.position;
36
37 float journey = 0f;
38
39 while (journey <= 1f)
40 {
41     journey += Time.deltaTime * speed;
42
43     model.transform.position =
44         Vector3.Lerp(startPosition, endPosition, journey);
45
46     yield return null;
47 }
48
49 currentTarget = (currentTarget + 1) % ImageTargets.Length;
50 isMoving = false;
51 }
52
53 1 reference
54 private ObserverBehaviour GetNextDetectedTarget()
55 {
56     foreach (ObserverBehaviour target in ImageTargets)
57     {
58         if (target != null &&
59             target.TargetStatus.Status == Status.TRACKED)
60         {
61             return target;
62         }
63     }
64
65     return null;
66 }
```

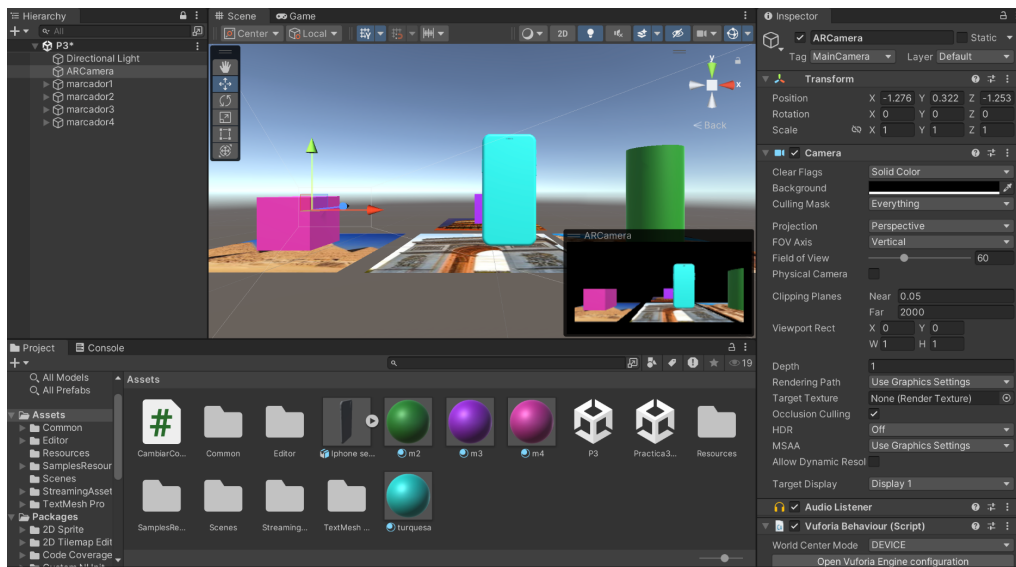
8. Correcciones realizadas al script

Durante la implementación se revisó la sintaxis para asegurar que Unity pudiera compilar el script y mostrar el método público dentro del evento On Click(). La versión final utiliza StartCoroutine(), Time.deltaTime y UnityEngine.Vector3. También se validó el estado del target con la enumeración Status de Vuforia y se empleó yield return null dentro del ciclo de interpolación.

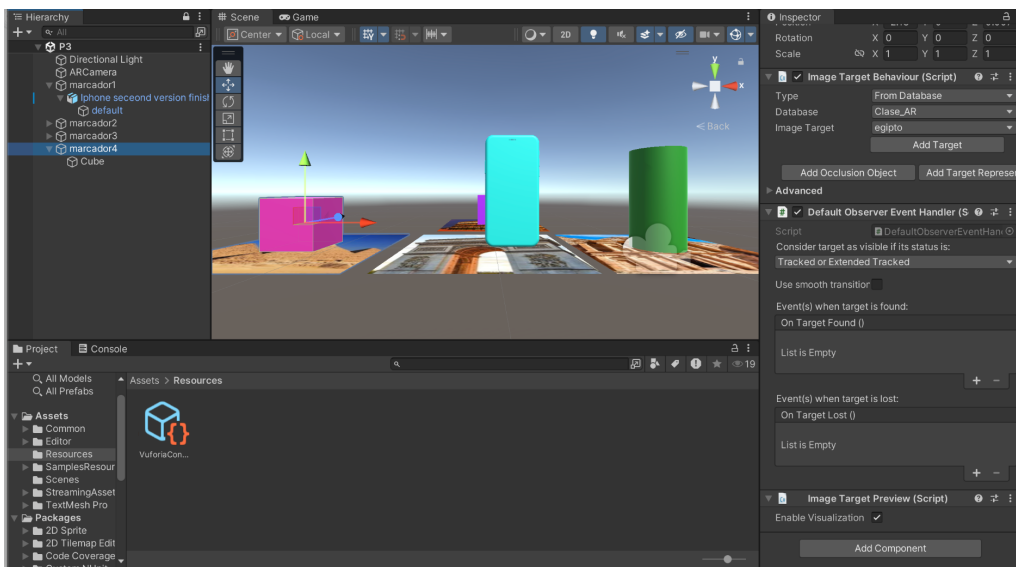
- StartCoroutine() se utiliza para iniciar correctamente la corrutina.
- Time.deltaTime controla el avance del movimiento por cuadro.
- Vector3 corresponde al tipo de UnityEngine.
- El flujo se detiene únicamente cuando no existe un target detectado.
- yield return null permite continuar la interpolación durante varios frames.
- El método moveToNexMarker() quedó visible y seleccionable en el evento On Click().

9. Prueba de funcionamiento en Unity

Con la escena configurada se ejecutó Play Mode y se utilizaron las imágenes registradas en la base Clase_AR. La cámara reconoció los marcadores y mostró sus respectivos objetos 3D. El botón de la interfaz quedó enlazado con move.moveToNexMarker(), permitiendo ejecutar el comportamiento definido en el script.



Vista general de la escena con los objetos asociados a los marcadores.

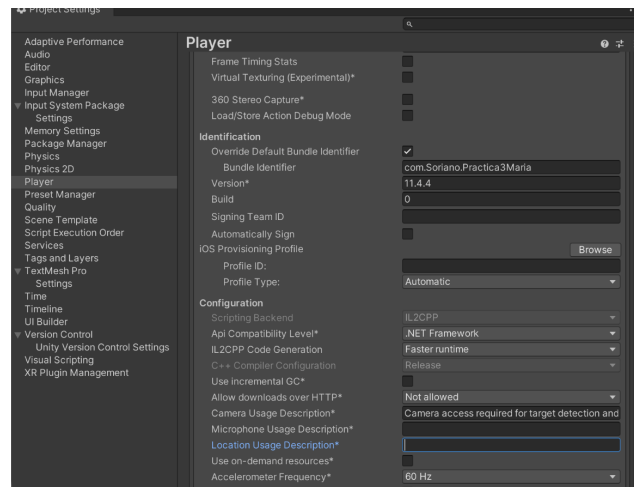


Configuración de la escena durante las pruebas de interacción.

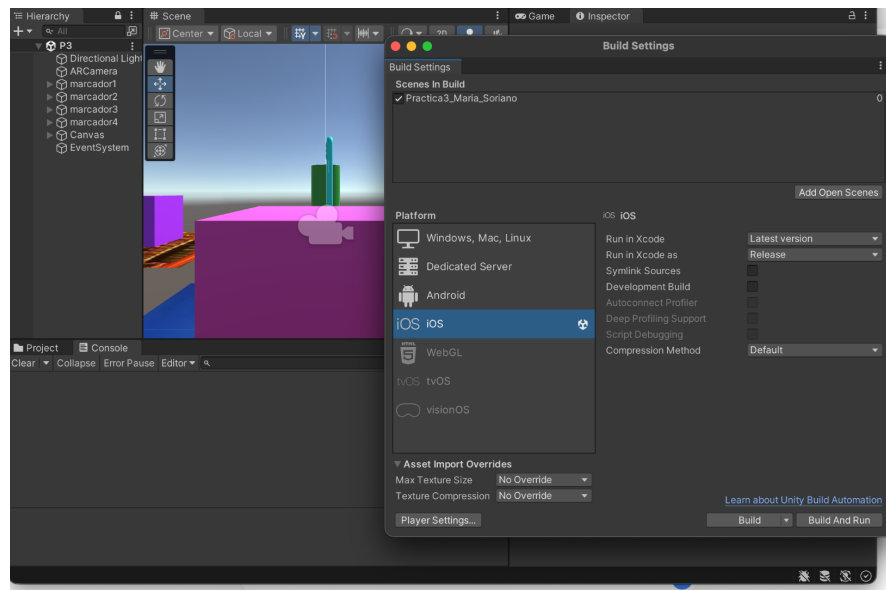
10. Preparación del proyecto para iOS

Para preparar la aplicación para iPhone se abrió File → Build Settings y se seleccionó iOS como plataforma. En Scenes In Build se dejó únicamente la escena final Practica3_Maria_Soriano. Después, desde Project Settings → Player, se actualizó la identidad de la aplicación para diferenciarla de las prácticas anteriores.

- Product Name: Practica3_Maria_Soriano.
- Bundle Identifier: com.Soriano.Practica3Maria.
- Minimum iOS Version: 15.0.
- Camera Usage Description configurada para permitir el acceso a la cámara.
- Orientación principal configurada en Portrait.



Configuración de identificación y parámetros de iOS en Player Settings.

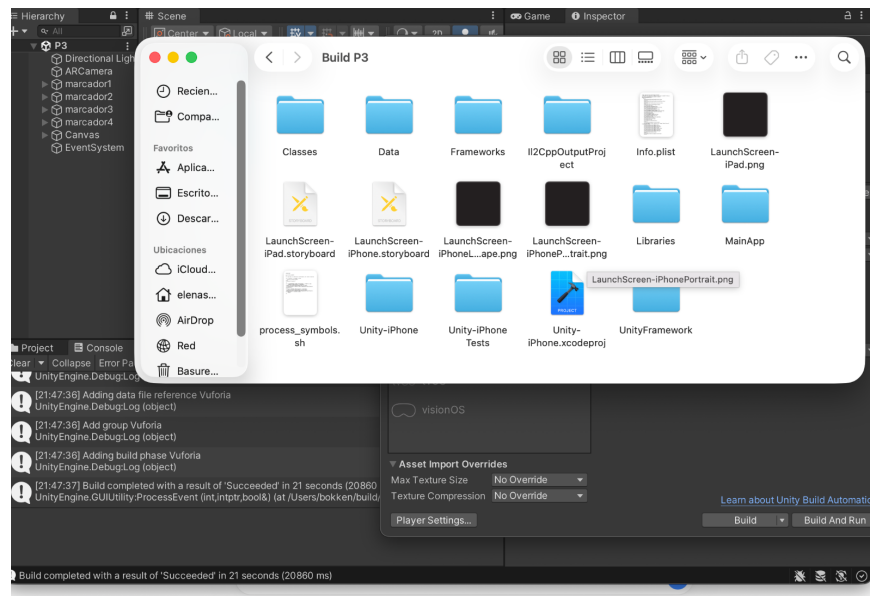


Escena final seleccionada en Build Settings para iOS.

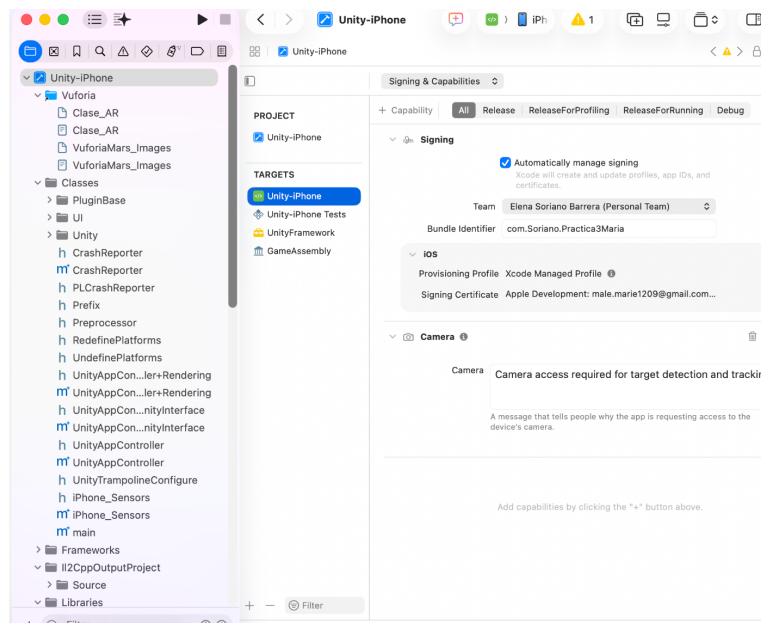
11. Generación del proyecto y firma en Xcode

Unity generó correctamente el proyecto de Xcode en una carpeta de salida independiente. Entre los archivos creados se incluyeron Unity-iPhone.xcodeproj, UnityFramework, Classes, Data, Libraries e Info.plist. Posteriormente se abrió Unity-iPhone.xcodeproj en Xcode y se seleccionó el target Unity-iPhone.

En Signing & Capabilities se activó Automatically manage signing y se seleccionó el Personal Team. Xcode creó el provisioning profile y el certificado de desarrollo. El iPhone físico se seleccionó como dispositivo de destino para ejecutar e instalar la aplicación.



Carpeta de salida generada por Unity para iOS.



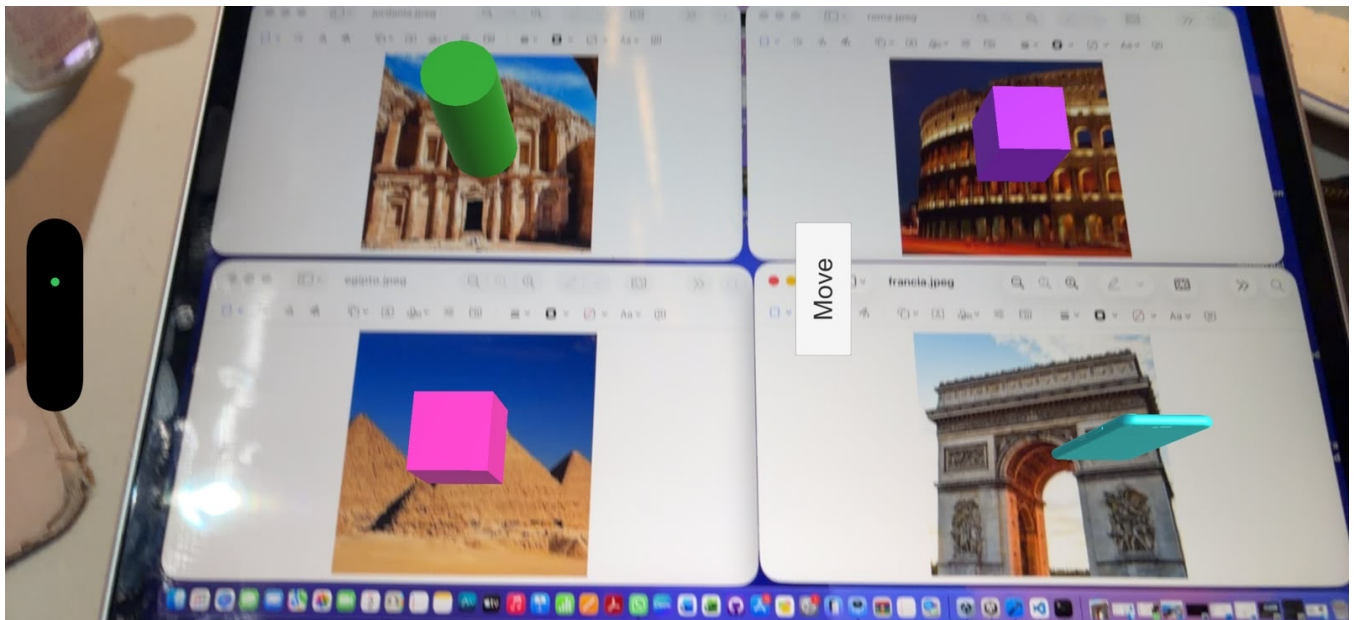
Firma automática configurada en Xcode con Personal Team.

Resultados

La Práctica 3 permitió ampliar una escena de realidad aumentada para trabajar con varios Image Targets y diferentes objetos 3D. Dentro del Editor de Unity se comprobó la detección de los marcadores, la aparición de los modelos y la integración del botón con el método `moveToNexMarker()`. Asimismo, se generó correctamente el proyecto para iOS y se completó la configuración de firma en Xcode.

- Se configuraron cuatro Image Targets dentro de una misma escena.

- Cada marcador quedó asociado a un target de la base Clase_AR.
- Se utilizaron objetos 3D diferentes para comprobar el reconocimiento.
- Se crearon y aplicaron materiales de distintos colores.
- Se configuró Vuforia para trabajar con hasta cuatro targets simultáneos.
- Se agregó una interfaz Canvas con un botón funcional.
- El botón quedó conectado a `move.moveToNextMarker()`.
- El script compiló correctamente y pudo ejecutarse desde la interfaz.
- El proyecto iOS se generó con un Bundle Identifier independiente.
- Xcode quedó configurado con firma automática y dispositivo físico.



Resultados del funcionamiento

Repositorio de GitHub

Para mantener el proyecto organizado y conservar el código fuente, la escena, los scripts, los recursos de Vuforia y los materiales utilizados en la práctica, se integró la Práctica 3 al repositorio de GitHub de la materia. El repositorio funciona como respaldo del desarrollo y permite consultar los archivos del proyecto y sus avances.

Enlace de GitHub: https://github.com/mariaelena-soriano/TSIC3_RealidadAumentada

https://github.com/mariaelena-soriano/TSIC3_RealidadAumentada/tree/main/Practica3_Maria_Soriano

Evidencias en video

Para documentar el funcionamiento de la práctica se consideran dos evidencias: una grabación desde la pestaña Game de Unity y una segunda grabación del resultado final. De esta forma se muestra primero la validación del reconocimiento y la interacción dentro del Editor, y después el comportamiento final de la práctica.

Funcionamiento desde Game (Unity)

En esta evidencia se debe observar la ejecución en Play Mode desde la pestaña Game, el reconocimiento de los Image Targets, la aparición de los objetos 3D asociados y la interacción con el botón configurado para ejecutar `moveToNexMarker()`.

Enlace del video de funcionamiento en Game: <https://youtu.be/oseLQuBiLHQ>

Resultado final de la práctica

La segunda evidencia corresponde al resultado final de la Práctica 3. Debe mostrar la escena completa en funcionamiento, los marcadores reconocidos, los objetos 3D visibles sobre sus respectivos targets y la respuesta del botón de movimiento. Si se utiliza el iPhone como evidencia final, también puede mostrarse la aplicación instalada y ejecutándose en el dispositivo.

Enlace del video del resultado final: <https://youtu.be/0dTnnF4qYWw>

Referencias

Unity Technologies. (2023). Unity User Manual 2022.3 LTS.

<https://docs.unity3d.com/2022.3/Documentation/Manual/>

Unity Technologies. (2023). Coroutines. Unity Manual.

<https://docs.unity3d.com/2022.3/Documentation/Manual/Coroutines.html>

Unity Technologies. (2023). Vector3.Lerp. Unity Scripting API.

<https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Vector3.Lerp.html>

PTC. (2026). Vuforia Engine Developer Library. <https://developer.vuforia.com/library/>

Apple. (2026). Xcode documentation. <https://developer.apple.com/documentation/xcode/>